```
18      else if(s == 0) ret = 'B';
19      else ret = 'F';
20
21      // 如果只有一个节点，不需要递归了，直接打印
22      if(l == r)
23      {
24          cout << ret;
25          return;
26      }
27
28      // 先处理左右，再打印自己
29      int mid = (r + l) / 2;
30      dfs(l, mid);
31      dfs(mid + 1, r);
32      cout << ret;
33  }
34
35  int main()
36  {
37      cin >> n >> s;
38      n = (1 << n);
39      s = " " + s;
40      for(int i = 1; i <= n; i++)
41      {
42          if(s[i] == '1') sum[i] = sum[i - 1] + 1;
43          else sum[i] = sum[i - 1];
44      }
45      dfs(1, n);
46
47      return 0;
48  }
```

12. 分治

分治，字面上的解释是「分而治之」，就是把一个复杂的问题分成两个或更多的相同或相似的子问题，直到最后子问题可以简单的直接求解，原问题的解即子问题的解的合并。

12.1 逆序对

> **题目来源：** 洛谷
>
> **题目链接：** P1908 逆序对
>
> **难度系数：** ★★★

【题目描述】

> 猫猫 TOM 和小老鼠 JERRY 最近又较量上了，但是毕竟都是成年人，他们已经不喜欢再玩那种你追我赶的游戏，现在他们喜欢玩统计。
>
> 最近，TOM 老猫查阅到一个人类称之为“逆序对”的东西，这东西是这样定义的：对于给定的一段正整数序列，逆序对就是序列中 $a_i > a_j$ 且 $i < j$ 的有序对。知道这概念后，他们就比赛谁先算出给定的一段正整数序列中逆序对的数目。注意序列中可能有重复数字。

【输入描述】

> 第一行，一个数 n ，表示序列中有 n 个数。
>
> 第二行 n 个数，表示给定的序列。序列中每个数字不超过 10^9 。
>
> 对于所有数据， $n \leq 5 \times 10^5$

【输出描述】

> 输出序列中逆序对的数目。

【示例一】

> 输入：
>
> ```
> 6
> 5 4 2 6 3 1
> ```
>
> 输出：
>
> ```
> 11
> ```

【解法】

「分治」是解决「逆序对」非常经典的解法，后续我们也会学到利用「树状数组」或「线段树」解决逆序对问题。

如果把整个序列 $[l, r]$ 从中间 mid 位置分成两部分，那么逆序对个数可以分成三部分：

- $[l, mid]$ 区间内逆序对的个数 $c1$ ；
- $[mid + 1, r]$ 区间内逆序对的个数 $c2$ ；
- 从 $[l, mid]$ 以及 $[mid + 1, r]$ 各选一个数，能组成的逆序对的个数 $c3$

那么逆序对的总数就是 $c1 + c2 + c3$ 。其中求解 $c1, c2$ 的时候跟原问题是一样的，可以交给「递归」去处理（经过前面题目的铺垫，我相信大家应该能够理解），那我们重点处理「一左一右」的情况。

如果在处理一左一右的情况时，两个数组「无序」，我们的时间复杂度其实并没有优化到哪去，甚至还「不如暴力解法」。但是如果两个数组有序的话，我们就可以快速找出逆序对的个数。

先不管怎么求逆序对，我们能让左右两个数组有序嘛？当然可以，这不就是「归并排序」么。因此，我们能做到在求完 $c1, c2$ 之后，然后让「左右两个区间有序」。

那么接下来问题就变成，已知两个「有序数组」，如何求出左边选一个数，右边选一个数的情况下的逆序对的个数。核心思想就是找到右边选一个数之后，左边区间内「有多少个比我大的」。



定义两个「指针」扫描两个有序数组：此时会有下面三种情况：

1. $a[cur1] \le a[cur2]$： $a[cur1]$ 不会与 $[cur2, r]$ 区间内任何一个元素构成逆序对， $cur1++$；
2. $a[cur1] > a[cur2]$：此时 $[cur1, mid]$ 区间内所有元素都会与 $a[cur2]$ 构成逆序对，逆序对个数增加 $mid - cur1 + 1$，此时 $cur2$ 已经统计过逆序对了， $cur2++$；

重复上面两步，我们就可以在 $O(N)$ 的时间内处理完「一左一右」时，逆序对的个数。而且，我们会发现，这跟我们「归并排序的过程」是高度一致的。所以可以一边排序，一边计算逆序对的个数。

【参考代码】

代码块

```
#include <iostream>

using namespace std;
typedef long long LL;

const int N = 5e5 + 10;

int n;
int arr[N], tmp[N];

LL merge(int l, int r)
{
    if(l >= r) return 0;

    int mid = (l + r) / 2;
    // [l, mid] [mid + 1, r]
    LL ret = 0;
```

```cpp
    // 先去算算左右两部分逆序对的和，顺便给左右两边排排序
    ret += merge(l, mid); ret += merge(mid + 1, r);

    // 计算一左一右时，逆序对的个数
    int cur1 = l, cur2 = mid + 1, i = l;
    while(cur1 <= mid && cur2 <= r)
    {
        if(arr[cur1] <= arr[cur2])
        {
            tmp[i++] = arr[cur1++];
        }
        else // 右边元素比左边小时，产生逆序对
        {
            // 统计逆序对的个数
            ret += (mid - cur1 + 1);
            tmp[i++] = arr[cur2++];
        }
    }
    while(cur1 <= mid) tmp[i++] = arr[cur1++];
    while(cur2 <= r) tmp[i++] = arr[cur2++];

    for(int j = l; j <= r; j++) arr[j] = tmp[j];
    return ret;
}

int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++) cin >> arr[i];

    cout << merge(1, n) << endl;

    // for(int i = 1; i <= n; i++) cout << arr[i] << " ";

    return 0;
}
```

12.2 求第 k 小的数

> **题目来源：** 洛谷
>
> **题目链接：** 求第 k 小的数
>
> **难度系数：** ★★★

【题目描述】

> 输入 n （$1 \leq n < 5000000$ 且 n 为奇数）个数字 a_i $(1 \leq a_i < 10^9)$，输出这些数字的第 k 小的数。最小的数是第 0 小。
>
> 请尽量不要使用 `nth_element` 来写本题，因为本题的重点在于练习分治算法。

【示例一】

> 输入：
>
> ```
> 5 1
> 4 3 2 1 5
> ```
>
> 输出：
>
> ```
> 2
> ```

【解法】

【参考代码】

代码块

```cpp
#include <iostream>
#include <ctime>

using namespace std;

const int N = 5e6 + 10;

int n, k;
int a[N];

// [l, r] 之间随机返回一个数
int get_random(int l, int r)
{
    return a[rand() % (r - l + 1) + l];
}

int quick_select(int l, int r, int k)
{
    if(l == r) return a[l];

    // 随机选择基准元素
    int p = get_random(l, r);
    // 以 p 为基准划分三部分
    int x = l - 1, y = r + 1, i = l;
    while(i < y)
    {
        if(a[i] == p) i++;
```

```
28          else if(a[i] < p) swap(a[++x], a[i++]);
29          else swap(a[--y], a[i]);
30      }
31
32      // [l, x] [x + 1, y - 1] [y, r]
33      int c1 = x - l + 1, c2 = y - 1 - x, c3 = r - y + 1;
34      if(k <= c1) return quick_select(l, x, k);
35      else if(k <= c1 + c2) return p;
36      else return quick_select(y, r, k - c1 - c2);
37  }
38
39  int main()
40  {
41      srand(time(0));
42
43      scanf("%d%d", &n, &k);
44      k++;
45      for(int i = 1; i <= n; i++)
46      {
47          scanf("%d", &a[i]);
48      }
49      printf("%d\n", quick_select(1, n, k));
50
51      return 0;
52  }
```

12.3 最大子段和

题目来源： 洛谷

题目链接： P1115 最大子段和

难度系数： ★★

【题目描述】

给出一个长度为 n 的序列 a ，选出其中连续且非空的一段使得这段和最大。

【输入描述】

第一行是一个整数，表示序列的长度 n。

第二行有 n 个整数，第 i 个整数表示序列的第 i 个数字 a_i。

- 对于 40% 的数据，保证 $n \leq 1 \times 10^3$ 。
- 对于 100% 的数据，保证 $1 \leq n \leq 2 \times 10^5, -10^4 \leq a_i \leq 10^4$ 。

【输出描述】

> 输出一行一个整数表示答案。

【示例一】

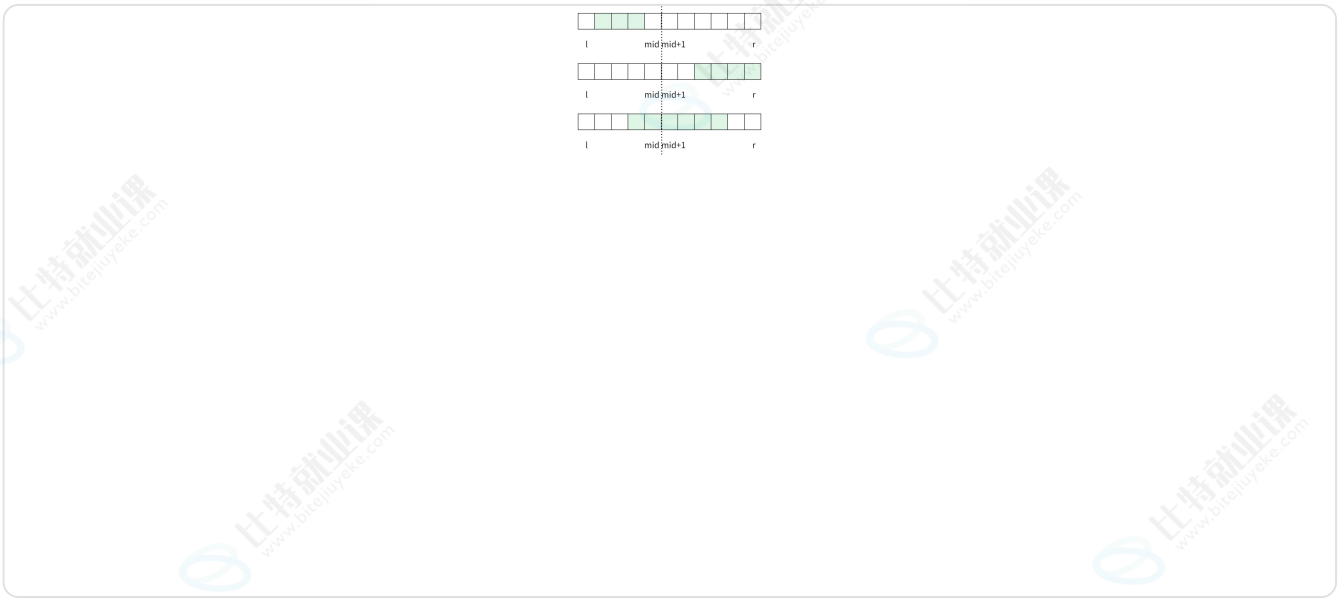
> 输入：
>
> 7
>
> 2 -4 3 -1 2 -4 3
>
> 输出：
>
> 4

【解法】

第三次遇见它了~

如果把整个序列 $[l,r]$ 从中间 mid 位置分成两部分，那么整个序列中「所有的子数组」就分成三部分：

- 子数组在区间 $[l,mid]$ 内；
- 子数组在区间 $[mid+1,r]$ 内；
- 子数组的左端点在 $[l,mid]$ 内，右端点在 $[mid+1,r]$ 内。



那么我们的「最终结果」也会在这三部分取到，要么在左边区间，要么在右边区间，要么在跨越中轴线的区间。因此，我们可以先求出左边区间的最大子段和，再求出右边区间的最大子段和，最后求出中间区间的最大子段和。其中求「左右区间」时，可以交给「递归」去解决。

那我们重点处理如何求出「中间区间」的最大子段和。可以把中间区间分成两部分：

- 左边部分是从 mid 为起点，「向左延伸」的最大子段和；

- 右边部分是从 $mid+1$ 为起点，「向右延伸」的最大子段和。

分别求出这两个值，然后相加即可。

求法也很简单，直接「固定起点」，一直把「以它为起点的所有子数组」的和都计算出来，取最大值即可。

【参考代码】

代码块

```
#include <iostream>

using namespace std;

const int N = 2e5 + 10;

int n;
int a[N];

int dfs(int l, int r)
{
    if(l == r) return a[l];

    int mid = (l + r) >> 1;
    // 先求一下左右两边的最大值
    int ret = max(dfs(l, mid), dfs(mid + 1, r));

    int suml = a[mid], sumr = a[mid + 1];
    int maxl = suml, maxr = sumr;
    // 求出以 a[mid] 开始，向左延伸的最大值
    for(int i = mid - 1; i >= l; i--)
    {
        suml += a[i];
        maxl = max(maxl, suml);
    }
    // 求出以 a[mid + 1] 开始，向右延伸的最大值
    for(int i = mid + 2; i <= r; i++)
    {
        sumr += a[i];
        maxr = max(maxr, sumr);
    }

    // 返回三种情况的最大值
    return max(maxr + maxl, ret);
}

```

```cpp
int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++) cin >> a[i];
    cout << dfs(1, n) << endl;

    return 0;
}
```

12.4 地毯填补问题

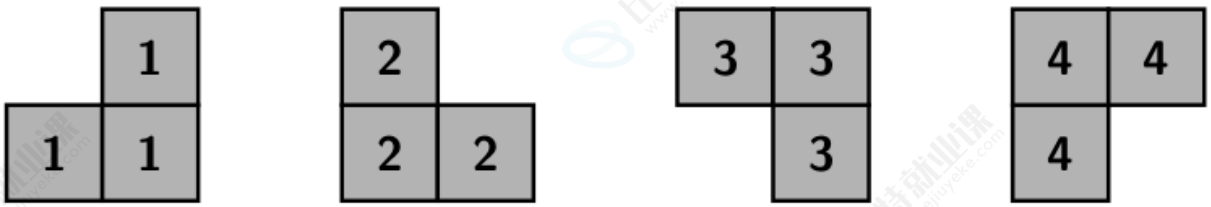
题目来源： 洛谷

题目链接： P1228 地毯填补问题

难度系数： ★★★

【题目描述】

相传在一个古老的阿拉伯国家里，有一座宫殿。宫殿里有个四四方方的格子迷宫，国王选择驸马的方法非常特殊，也非常简单：公主就站在其中一个方格子上，只要谁能用地毯将除公主站立的地方外的所有地方盖上，美丽漂亮聪慧的公主就是他的人了。公主这一个方格不能用地毯盖住，毯子的形状有所规定，只能有四种选择（如图）：



洛谷

并且每一方格只能用一层地毯，迷宫的大小为 $2^k \times 2^k$ 的方形。当然，也不能让公主无限制的在那儿等，对吧？由于你使用的是计算机，所以实现时间为 1 秒。

【输入描述】

输入文件共 2 行。

第一行一个整数 k，即给定被填补迷宫的大小为 $2^k \times 2^k (0 < k \leq 10)$；第二行两个整数 x, y，即给出公主所在方格的坐标（x 为行坐标，y 为列坐标），x 和 y 之间有一个空格隔开。

【输出描述】

将迷宫填补完整的方案：每一行为 x, y, c（x, y 为毯子拐角的行坐标和列坐标，c 为使用毯子的形状，具体见上面的图，毯子形状分别用 $1, 2, 3, 4$ 表示，x, y, c 之间用一个空格隔开）。

【示例一】

输入：

3

3 3

输出：

5 5 1

2 2 4

1 1 4

1 4 3

4 1 2

4 4 1

2 7 3

1 5 4

1 8 3

3 6 3

4 8 1

7 2 2

5 1 4

6 3 2

8 1 2

8 4 1

7 7 1

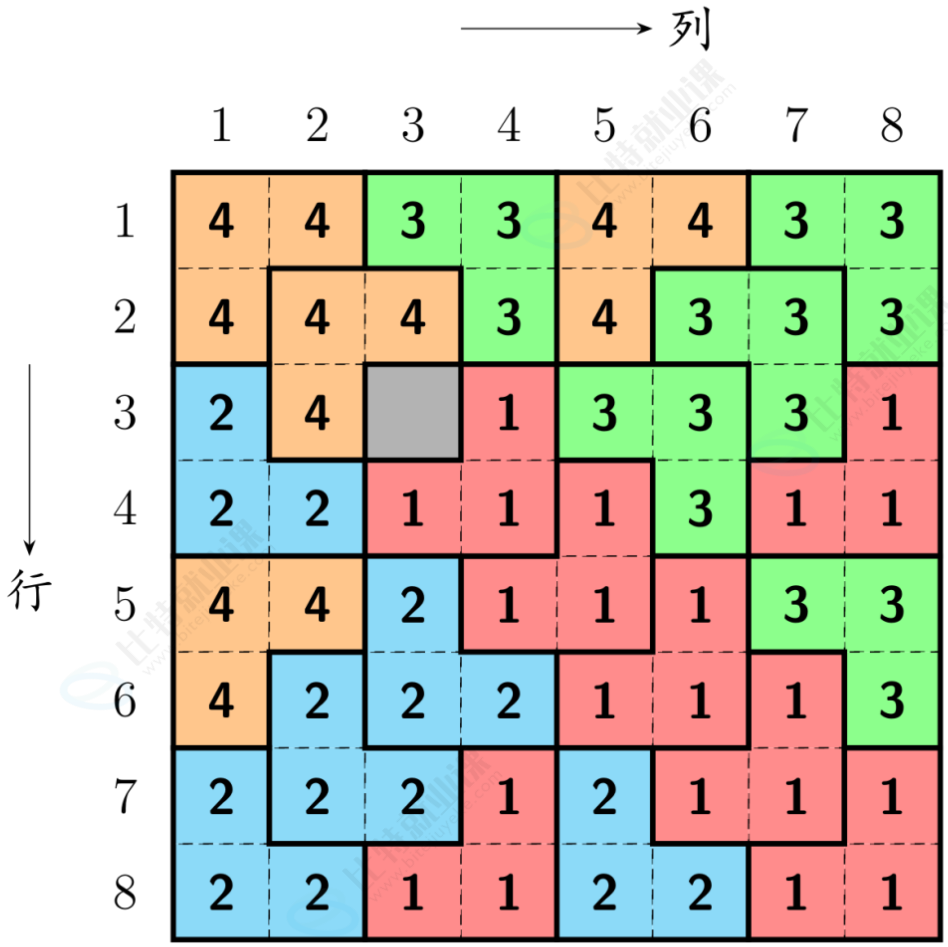
6 6 1

5 8 3

8 5 2

8 8 1

样例解释：

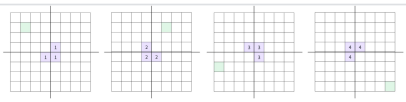


【解法】

非常经典的一道分治题目，也可以说是一道递归题目。

一维分治的时候，我们是从中间把整个区间切开，分成左右两部分（其实有时候我们可以三等分，就看具体问题是什么）。二维的时候，我们可以横着一刀竖着一刀，分成左上、右上、左下、右下四份。而这道题的矩阵长度正好是 2^k ，能够被不断平分下去。像是在暗示我们，要用分治，要用分治，要用分治.....

当我们把整个区间按照中心点一分为四后，四个区间里面必然有一个区间有缺口（就是公主的位置），那这四个区间不一样，那就没有相同子问题了。别担心，只要我们在中心位置放上一块地毯，四个区间就都有一个缺口了。如下图所示：



无论缺口在哪里，我们都可以在缺口对应的区间的角落，放上一个地毯。接下来四个区间都变成只有一个缺口的形式，就可以用递归处理子问题。

因此，我们拿到一个矩阵后的策略就是：

- 先四等分；
- 找出缺口对面的区间，放上一块地毯；
- 递归处理四个子问题

【参考代码】

代码块

```
#include <iostream>

using namespace std;

void dfs(int a, int b, int x, int y, int k)
{
    if(k < 1) return;
    int len = 1 << (k - 1);

    if(x < a + len && y < b + len) // 障碍物在左上角
    {
        cout << a + len << " " << b + len << " 1" << endl;
        dfs(a, b, x, y, k - 1);
        dfs(a, b + len, a + len - 1, b + len, k - 1);
        dfs(a + len, b, a + len, b + len - 1, k - 1);
        dfs(a + len, b + len, a + len, b + len, k - 1);
    }
    else if(x >= a + len && y >= b + len) // 障碍物在右下角
    {
        cout << a + len - 1 << " " << b + len - 1 << " 4" << endl;
        dfs(a, b, a + len - 1, b + len - 1, k - 1);
        dfs(a, b + len, a + len - 1, b + len, k - 1);
        dfs(a + len, b, a + len, b + len - 1, k - 1);
```

```cpp
        dfs(a + len, b + len, x, y, k - 1);
    }
    else if(x >= a + len) // 障碍物在左下角
    {
        cout << a + len - 1 << " " << b + len << " 3" << endl;
        dfs(a, b, a + len - 1, b + len - 1, k - 1);
        dfs(a, b + len, a + len - 1, b + len, k - 1);
        dfs(a + len, b, x, y, k - 1);
        dfs(a + len, b + len, a + len, b + len, k - 1);
    }
    else // 障碍物在右上角
    {
        cout << a + len << " " << b + len - 1 << " 2" << endl;
        dfs(a, b, a + len - 1, b + len - 1, k - 1);
        dfs(a, b + len, x, y, k - 1);
        dfs(a + len, b, a + len, b + len - 1, k - 1);
        dfs(a + len, b + len, a + len, b + len, k - 1);
    }
}

int main()
{
    int k, x, y; cin >> k >> x >> y;
    dfs(1, 1, x, y, k);

    return 0;
}
```